

Session 2: Integrating OpenAI APIs into n8n

Chapter 4: Data Manipulation, Dynamic Mapping, and "Execute Command".....	1
1. File Download: Introduction to "Dynamic Mapping".....	1
2. Unlocking the "Execute Command" Node.....	4
3. Executing Commands in the Operating System.....	5
4. System Exits (Exit Codes).....	7
5. Error Routing.....	8
✓ Solutions and Answers for Trainer.....	9

Chapter 4: Data Manipulation, Dynamic Mapping, and "Execute Command"

 In this chapter, we make the transition from "listening" to "action." In the world of DevOps and QA, it's not enough to know that a new document has appeared; we need to process it. We will learn how to pass data from one node to another (Dynamic Mapping) and how to use the most powerful node in n8n to execute commands directly in the operating system.

So far, our first node (the Trigger) has detected that you have uploaded a PDF or text file (e.g. test plan) to your Google Drive folder. But the Trigger node only *detect* the event and exposes data about it (name, format, ID), it does not actually download the file to your hard drive.

1. File Download: Introduction to "Dynamic Mapping"

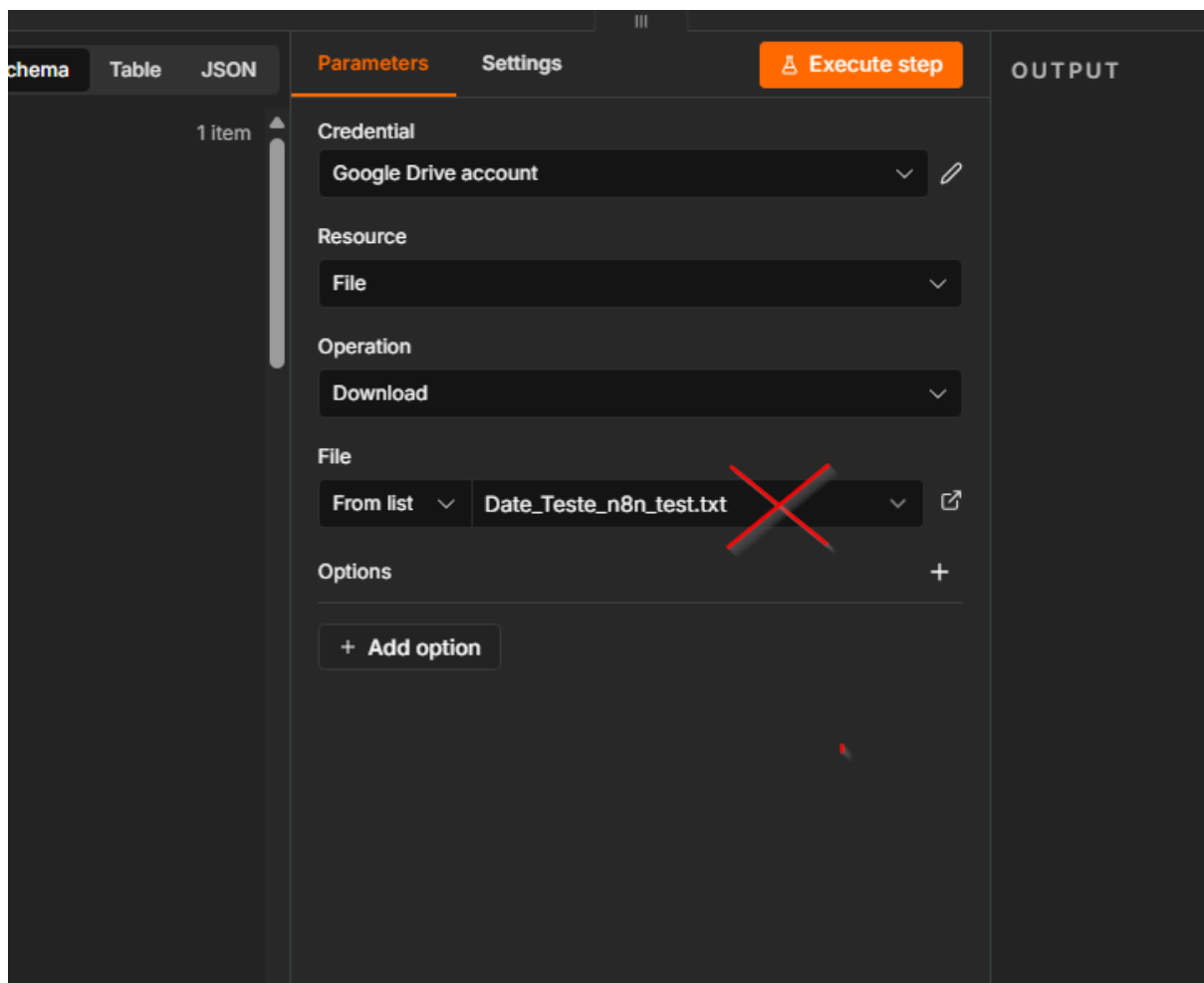
To bring the document to our local machine, we will add a second "robotic arm" to the flow.

- On the canvas, click on the little sign + (or draw a line from the green dot) to the right of the Trigger node created previously. Search for and add the application **Google Drive**.
- In the right pane, the section **Parameters** , we will configure exactly the desired action.

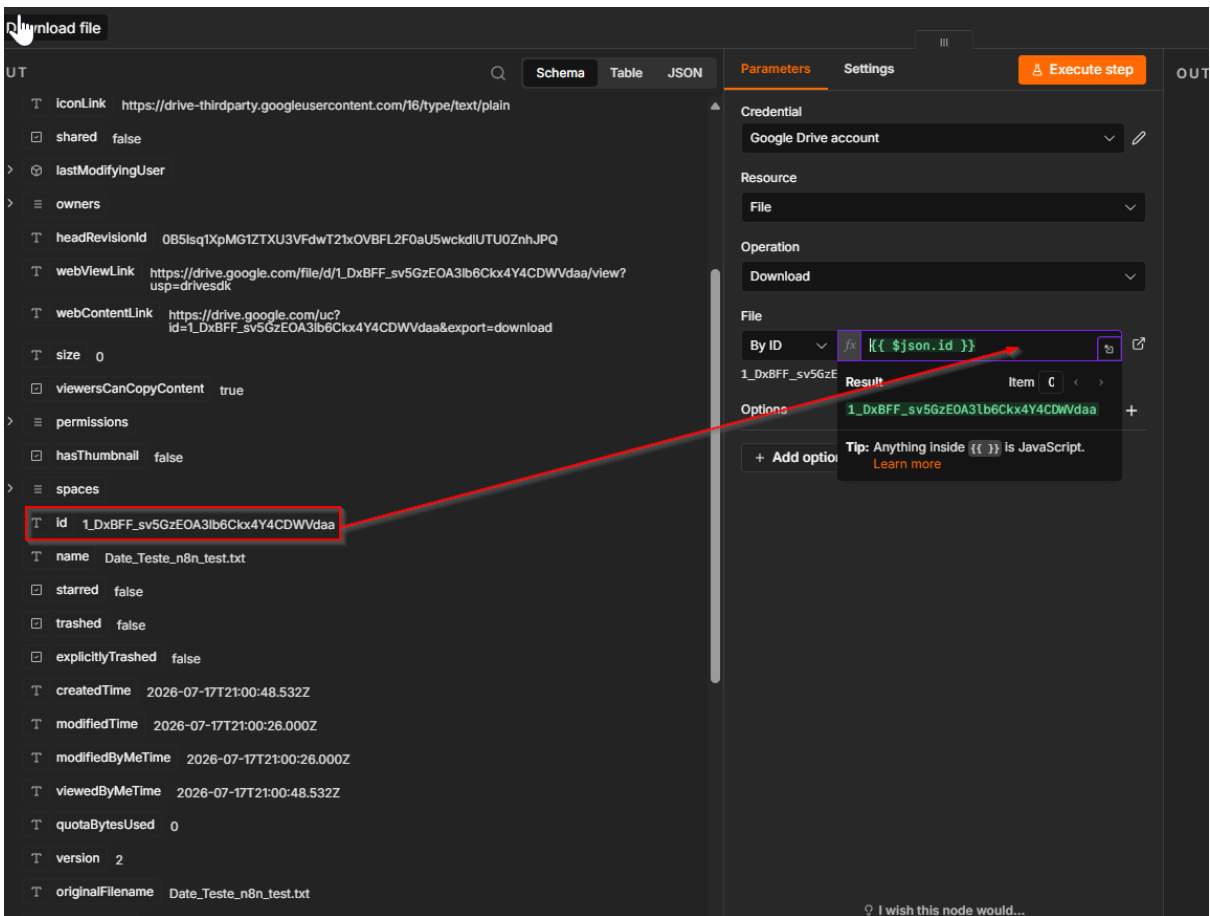
Make sure the settings are as follows:

- **Resource:** Choose **File**
- **Operation:** Choose **Download**
- Here comes a trap for beginners in the field **File**. By default, n8n gives you the option *From list* to choose a file by hand (as seen in our screenshot).

Warning! As automation engineers, we will never select a fixed file from that list ("hardcode"). If we did that, on every future run, the node would endlessly download the same old file, ignoring new documents.

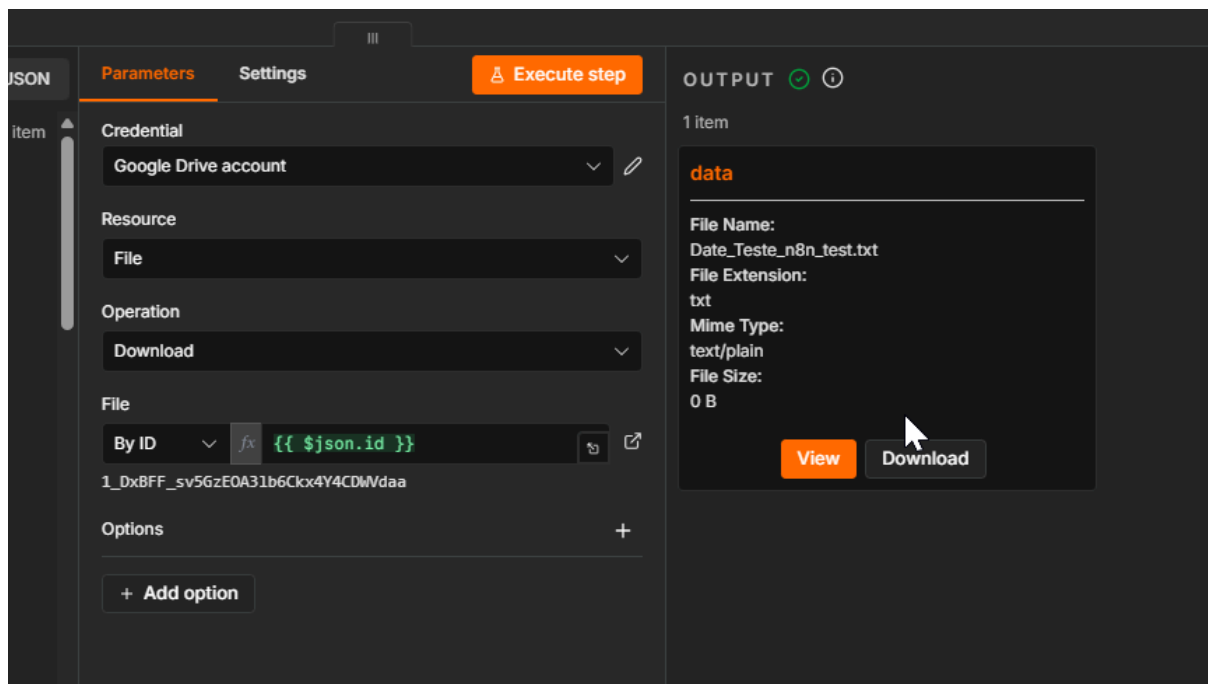


- **Dynamic Mapping (Drag & Drop):** Look in the panel on the left of the screen, called **INPUT**. There you see all the data exposed by the previous Trigger (such as mimeType, parents, etc.). Scroll down until you find the id field (the unique ID of the new file). Grab this field with your mouse and drag it (Drag & Drop) directly over the field **File** from the right panel, replacing manual selection.



- n8n will generate a dynamic expression like `{{ $json.id }}`. This tells the platform: "On each run, look at the current event and download the file detected at that exact second".

Press the orange button **Execute step** (visible above or on the right). Your file has been temporarily downloaded to the n8n platform's memory!



2. Unlocking the "Execute Command" Node

To write logs or run test scripts on your computer, n8n has a supreme node: **Execute Command**. It allows you to run Terminal commands (Bash, CMD, PowerShell) directly from the visual interface.

Why can't you find it in n8n search?

Starting with version 2.0, for strict security reasons (to prevent accidental deletion of system files), n8n has hidden this node from standard installations. To use it, we need to tell the platform that we know what we are doing, becoming true SysAdmins.

How to activate it (DevOps troubleshooting):

- Go to your Terminal (where n8n is running in the background) and stop the server by pressing **Ctrl + C**.
- We need to set a new Environment Variable called **NODES_EXCLUDE** (which implicitly blocks dangerous nodes) to an empty value `[]`. Run the commands below (don't forget to re-apply the SSL security variable from Chapter 2):
- **On Mac/Linux:**

```
export N8N_SECURE_COOKIE=false
```

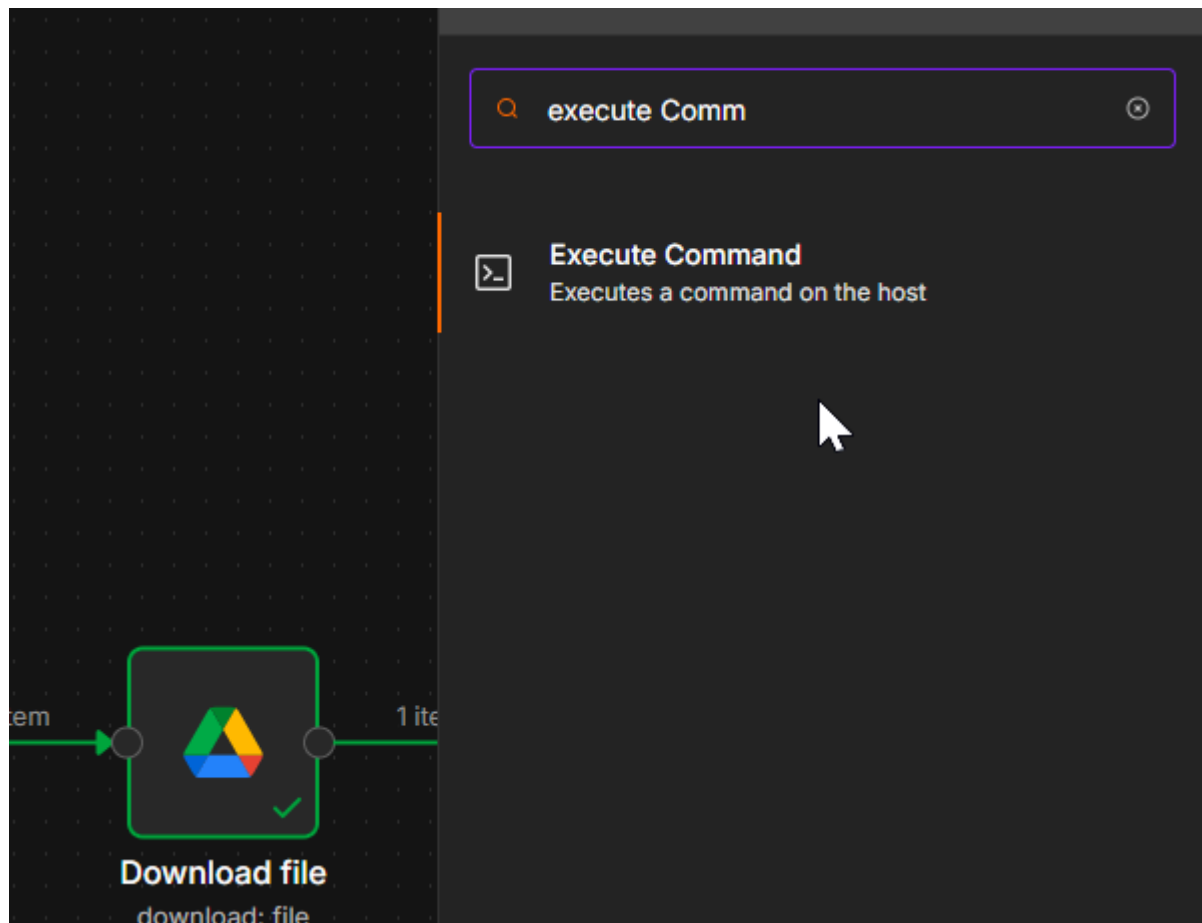
```
export NODES_EXCLUDE="[]"
```

```
n8n
```

- **Pe Windows (PowerShell):**

```
$env:N8N_SECURE_COOKIE="false"  
$env:NODES_EXCLUDE="[]"  
n8n
```

Once n8n has restarted, refresh the page in your browser. If you add a new step, the node **Execute Command** is now unlocked and ready for action!



3. Executing Commands in the Operating System

The n8n platform now has full access to your machine. We will demonstrate this by creating a log file directly on your hard drive in which we will note the name of each downloaded test

plan.

- Click on the sign again to the right of the Download File node and look for the newly unlocked node, **Execute Command**.
- In the field **Command** , type the beginning of your terminal command: echo "File downloaded:
- Now, use the left panel (Input) again to drag the name parameter (the original file name) directly inside your command, after the space.
- Continue the command by closing the quotes and adding the write operator to a text file: " >> n8n_logs.txt
- Your final expression will look exactly like this:

```
echo "File downloaded: {{ $json.name }}" >> n8n_logs.txt
```

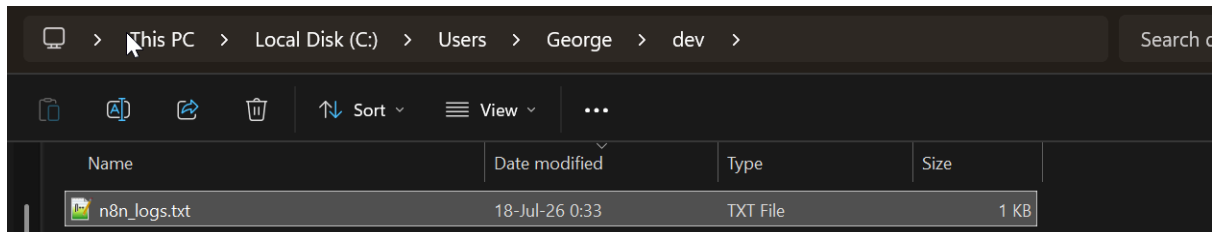
💡 DevOps Trap: Where was the file saved?

If you pressed **Execute step**, n8n ran the command silently. But where is the n8n_logs.txt file?

He was saved in *Current Working Directory (CWD)* That is, exactly in the folder where your Terminal was open when you started the server with the n8n

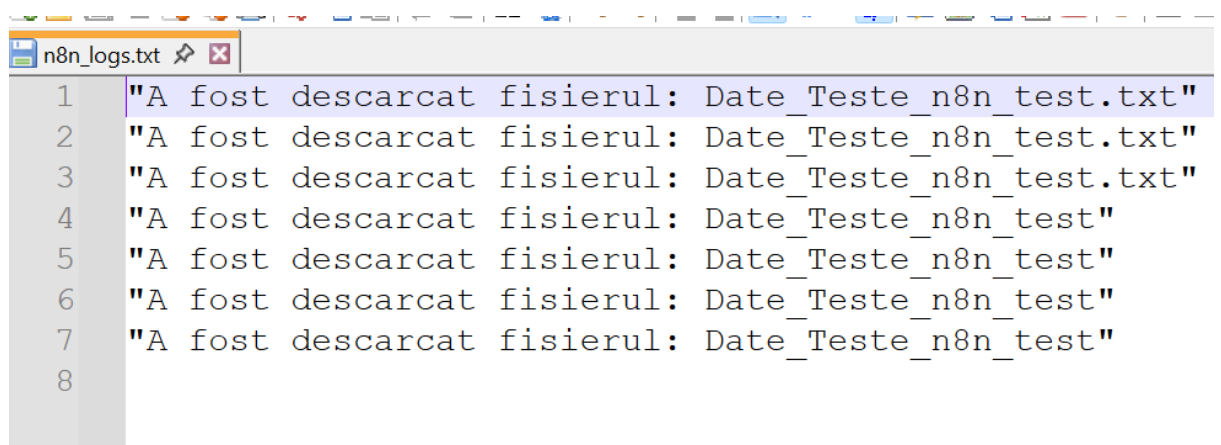
command.

Want to know the exact path? Temporarily delete the command with echo and run the command cd (on Windows) or pwd (on Mac). In the Output panel you will see the exact address of the folder where your log is located printed!



💡 Architect's Eye (Data Flow): Where are my logs?!

If you open the n8n_logs.txt file you just found, you'll notice something strange: it only contains the text we wrote ("File downloaded..."). Where are the thousands of lines of text inside the original document you downloaded? The answer is essential to understanding n8n: Our echo command only used the \$json.name parameter, which is a "tag" (Metadata). We didn't tell the system to read the file. But don't worry, the downloaded file wasn't lost! The Google Drive node took all that massive content and stored it safely directly in **n8n Platform RAM (Binary Data)**. The platform keeps the information "on the pipe", hidden from the hard drive, being perfectly prepared to send it to Artificial Intelligence in the next chapter!



4. System Exits (Exit Codes)

After executing the command node, in the results panel on the right you will notice a vital technical field: exitCode.

In computer science, any command executed on an operating system reports a status upon completion:

- **Exit Code 0:** It means complete success. The command ran without any errors.
- **Exit Code 1 (or any other number greater than 0):** It means that an error occurred (e.g. missing permissions, nonexistent folder, wrong syntax, failed tests).

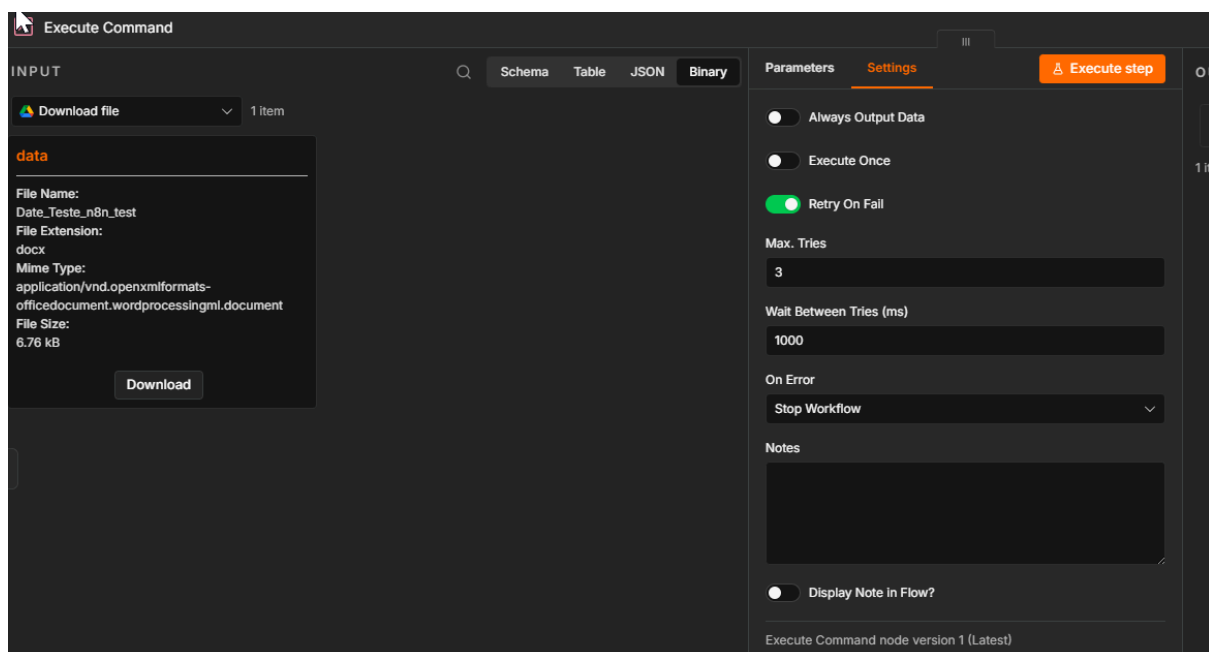
This exitCode will become our most important parameter in the following chapters. We will send it to the Artificial Intelligence to decide whether to alert the team.

5. Error Routing

Let's move on to a pure DevOps scenario. What happens if a script crashes, the terminal returns a system error, and the "Execute Command" node fails?

By default, n8n stops the entire stream and marks it red. But we want to be proactive, we want alerts!

- In any node window (including Execute Command), click the tab **Settings** (next to Parameters).
- Find the option **Continue On Fail** (Continue on failure) and activate it.
- Now, even if the node fails, the flow continues, and a special field called error will appear in the JSON output. This field becomes vital for creating smart alerts in the future.



💡 Did you know that...?

💡 Can the capabilities of the "Execute Command" node be extended beyond the local server? Using the twin node called **SSH** , you can orchestrate executions on remote machines. Imagine the downloaded file as a configuration script; via n8n and SSH, you can run commands that automatically build an entire new testing infrastructure in the cloud!

🧱 Applied Story: "The Puppeteer and the Marionette"

- **Traditional scripting:** It's like you're on a stage and manually moving each limb of the puppet (the operating system) by typing line by line. If you stop typing, the puppet stops.
- **Execute Command in n8n:** it doesn't become *puppeteer* from behind the curtain. He has the wires already tied to the puppet. All he does is wait for a signal from outside (the Trigger) and he automatically pulls the wires, executing the command exactly when and how he was instructed, while you, as the engineer, are in the audience, just supervising the show.

👤 Thought Exercise: "The Traffic Policeman"

You have just configured an Execute Command node that starts a Selenium automated test suite on a local server. The command has completed, and the output panel returns `exitCode: 130`.

Your task: Without writing any code, how would you visually configure the next step in n8n to ensure your team receives an alert email *only* if the tests fail, and is not bothered at all if the tests pass successfully (exit code 0)?

✅ Solutions and Answers for Trainer

Solution to the "Traffic Policeman" exercise:

To evaluate the result and make a decision based solely on the `exitCode`, we need to use a logical routing node.

- Immediately after the node *Execute Command* , we will add a node of type **IF** (or *Switch*).
- We will configure the condition inside the IF node to extract the dynamic value {{

```
$json.exitCode }}.
```

- The logical rule will be: **IF** exitCode is *Equal to* (Equal) 0 -> Send the stream on the branch **True**. If different (ex: 130) -> Send the stream on the branch **False**.
- Add a new node to *Send Email* or *Slack* **ONLY** on the branch outlet **False**.

So, if the code is 0, the flow stops silently. If an error occurs, the flow takes the failure route and triggers the alert!